

COMP0008:
Computer Architecture and Concurrency

Live Session (week 7)

Stefano Vissicchio

Warmup

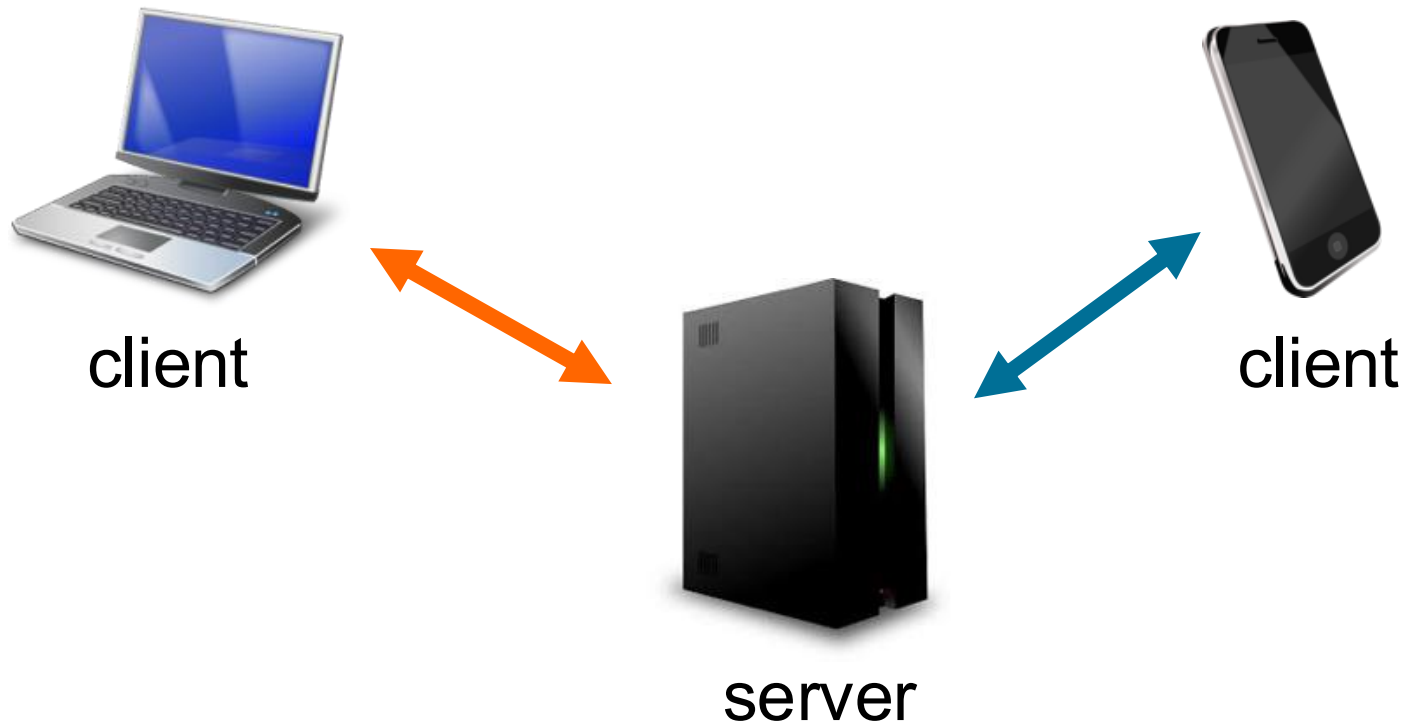
- Self-assess on www.menti.com w/ code 5207 6485
 - would help me try and tune review speed
- Interactions were great last week
 - let's keep going!
- Experiment: use Mentimeter to let you flag if I am too fast or too slow
 - same Mentimeter code as above

The concurrency story so far

- Program execution speed is important!
- Modern hardware uses multiple **techniques to support faster program execution**:
 - pipelining and instruction reordering
 - memory hierarchy and caching
 - superscalars, speculation, out-of-order execution, ...
- These techniques **don't come for free**
 - more complex hardware (and potentially compilers)
 - need to deal with data/control hazards
- Let's now take a programmer perspective

Suppose we are implementing a Web service

- We are asked to implement a server supporting Web-accessible functionalities
 - many clients can simultaneously ask for different services (e.g., involving mathematical computations)

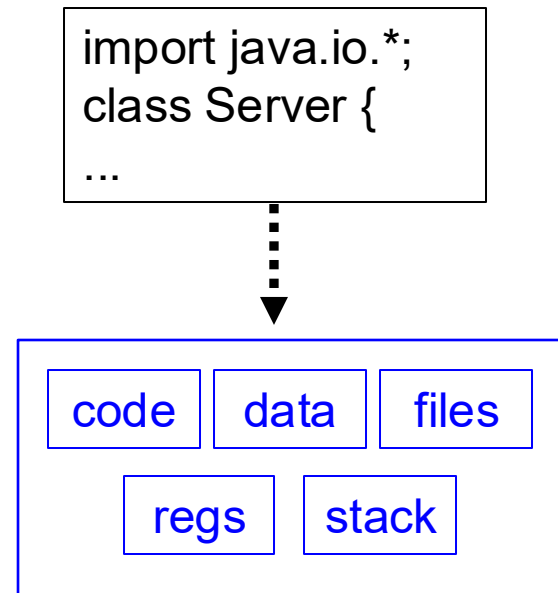


Suppose we are implementing a Web service

- We are asked to implement a server supporting Web-accessible functionalities
 - many clients can simultaneously ask for different services
- Processing clients' messages requires resources
 - some functionalities require long calculations and/or computations (CPU intensive)
 - other functionalities may need many memory reads/writes (I/O intensive) and network interactions (network intensive)
- How do we design the server-side application?

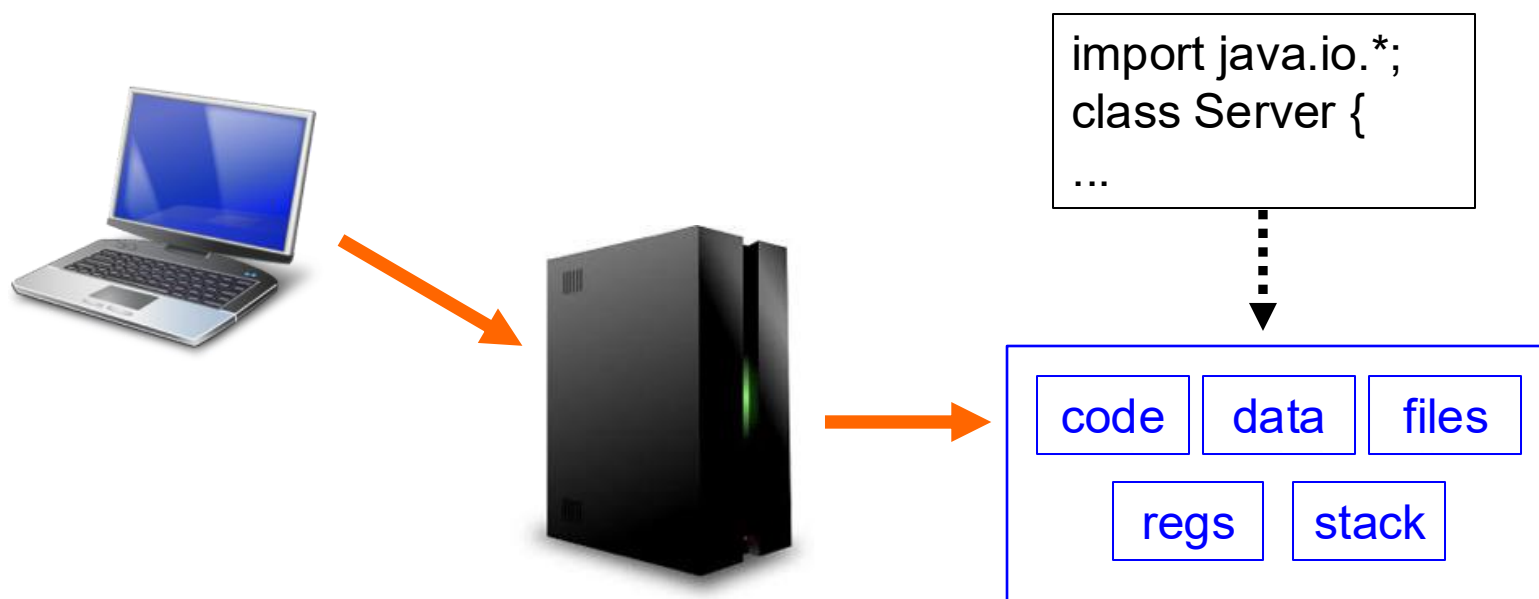
Design 1: single process

- We can write a monolithic program and let the operating system (OS) execute it as a **process**
 - behind the scenes, the OS allocates resources to run our process separately to the others running on the server



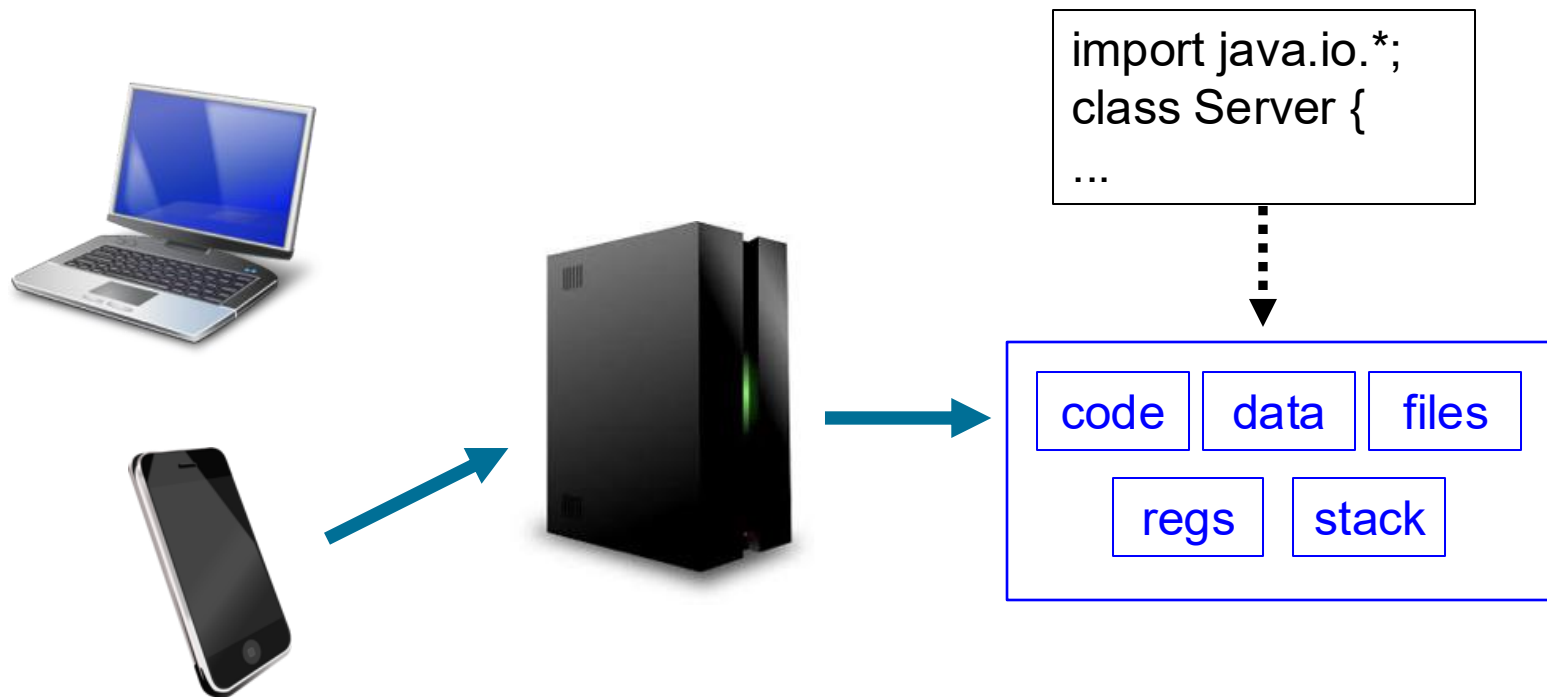
Design 1: single process

- We can write a monolithic program and let the operating system (OS) execute it as a **process**
 - behind the scenes, the OS allocates resources to run our process separately to the others running on the server



Design 1: single process

- We can write a monolithic program and let the operating system (OS) execute it as a **process**
 - behind the scenes, the OS allocates resources to run our process separately to the others running on the server



Design 1: single process

- When running the code in production, users complain it is **very slow**, and blocks at times. ***Why?***
 - we check the network, and it is not the bottleneck
 - we detect no server crush
 - we find no major bug in the code
- We isolate a **minimal version of our server** and a workload that exhibit the above problem
 - see demo

Design 1: single process

- Running our minimal server version with a few clients, we indeed note **high computation times**
 - Example

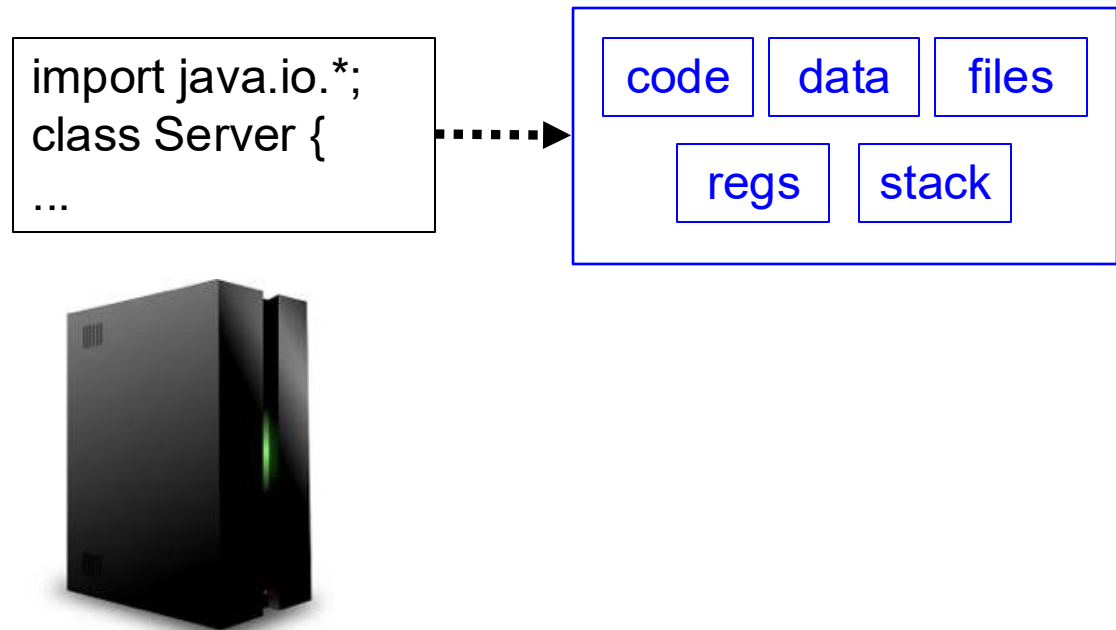
```
$ java SimulateServer
INFO: starting to serve clients
[Client 1] factorized 10 numbers
[Client 2] factorized 10 numbers
...
[Client 10] factorized 10 numbers
INFO: completed client requests in 15801 ms
```

Design 1: single process

- When running the code in production, users complain it is very slow, and blocks at times. *Why?*
- Problem: our program uses **only 1 of the 64 cores** of our server machine
 - our server process is run on one processor core, while the other cores remain idle
 - hence, a long computation to serve one client prevents the server to work for other clients
 - aside: can we **expose the above** with our minimal server?
- ***What to do?***

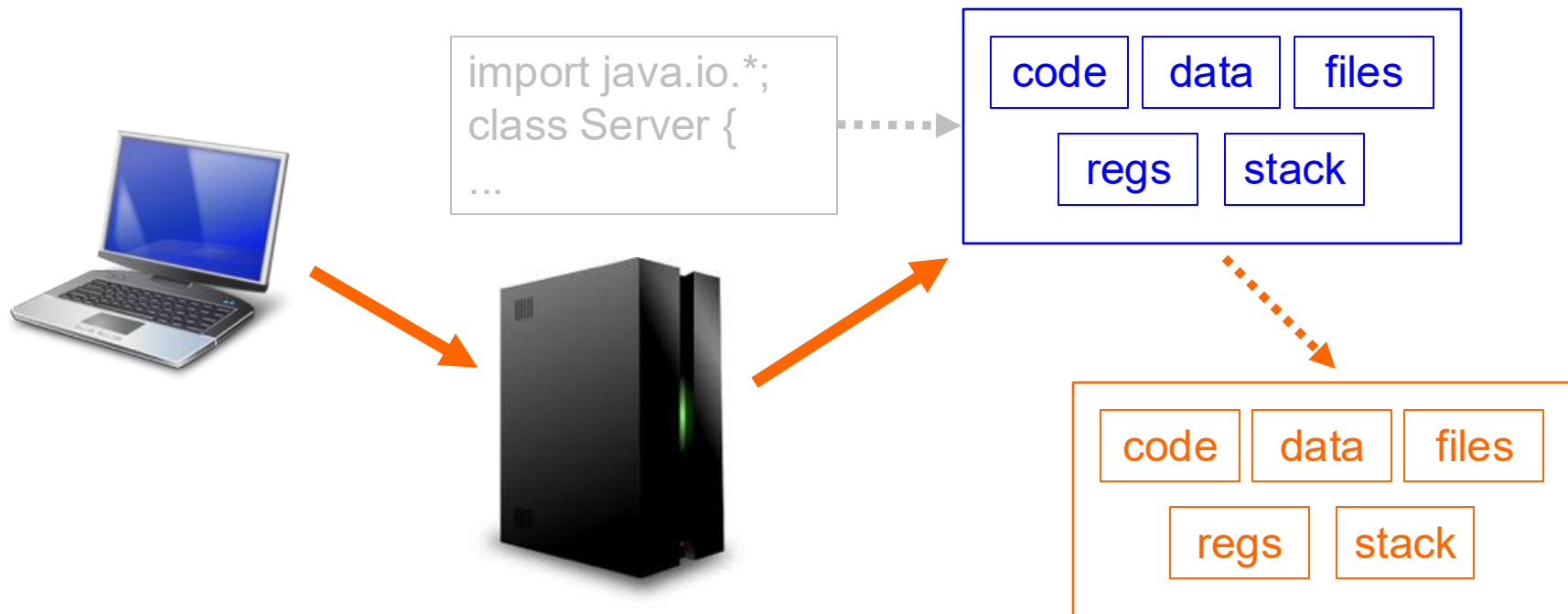
Design 2: multiple processes

- We can structure our program so that the OS runs multiple processes, e.g., one for each client
 - behind the scenes, the OS tries to balance the load across cores



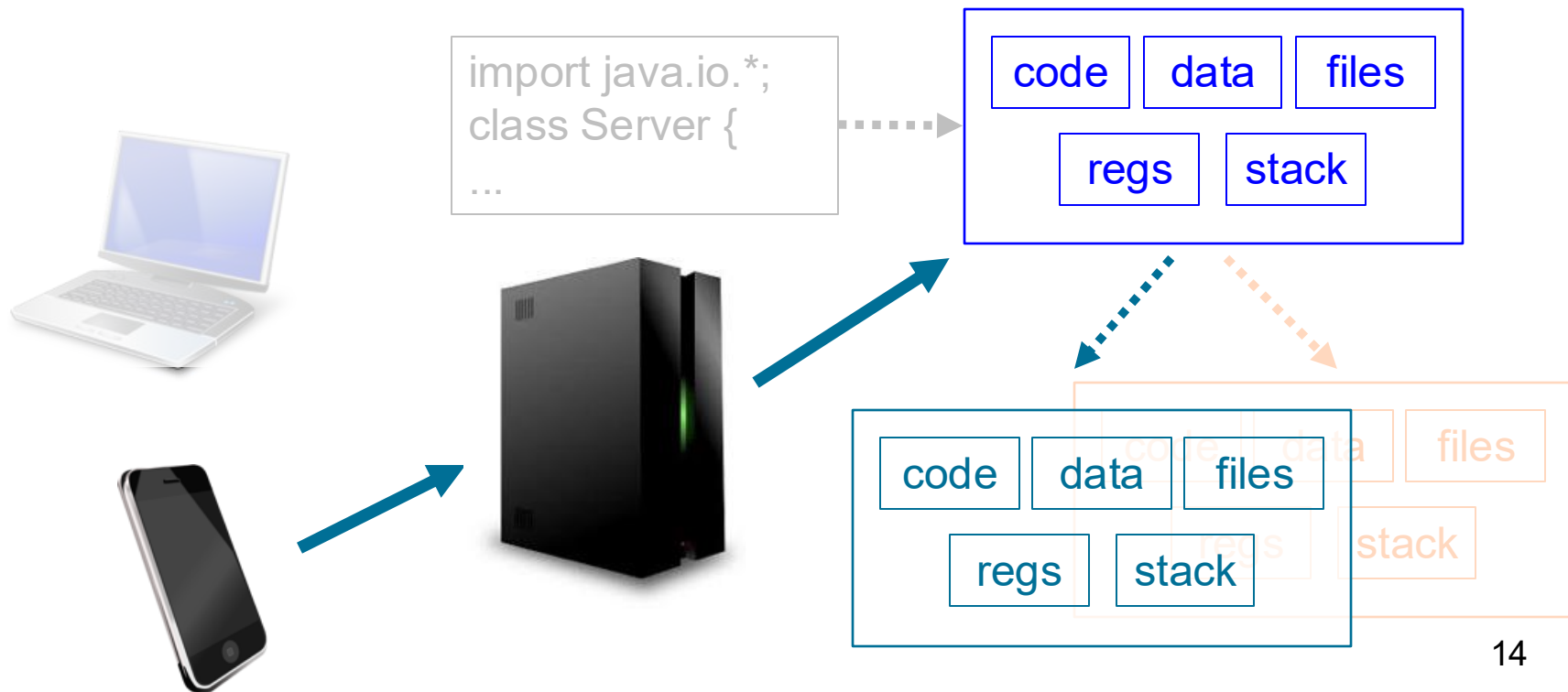
Design 2: multiple processes

- We can structure our program so that the OS runs multiple processes, e.g., one for each client
 - behind the scenes, the OS tries to balance the load across cores



Design 2: multiple processes

- We can structure our program so that the OS runs multiple processes, e.g., one for each client
 - behind the scenes, the OS tries to balance the load across cores



Design 2: multiple processes

- When running the code in production, sys admins complain that the application **over-utilizes servers.**

Why?

- our implementation looks reasonable and not wasteful in terms of resource utilisation
- Aside: can we reproduce the problem with our minimal server?

Design 2: multiple processes

- Running our minimal server version and isolating info from monitoring commands like top, we indeed note that **each process requires its resources**
 - Example

USER	PID	%CPU	%MEM	TIME	COMMAND
name	61754	78.1	0.4	0:01.33	java SimulateServer ..
name	61755	78.1	0.4	0:01.33	java SimulateServer ..
name	61758	78.1	0.5	0:01.33	java SimulateServer ..
...					
name	61745	0.0	0.4	0:00.10	java SimulateServer ..

Design 2: multiple processes

- When running the code in production, sys admins complain that the application over-utilizes servers.
Why?
- Problem: **processes are heavy** in terms of resource consumption
 - the OS allocates **separate resources to each process**: too many processes make resources scarce
 - **inter-process communication** is expensive too
 - in the worst case, the OS spends more time managing processes and allocated resources than anything else
- ***What now?***

Design 3: multiple threads

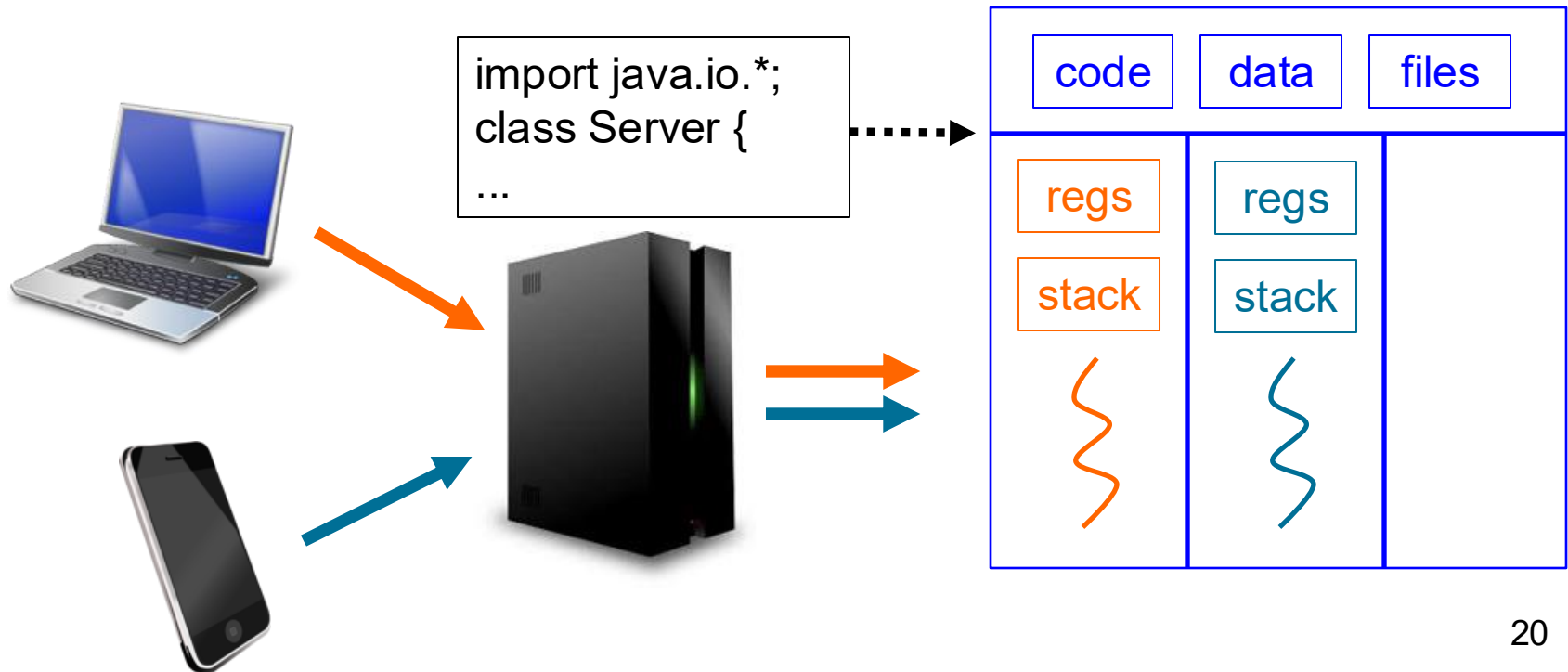
- We can re-factor our program so that the OS runs a *single process* which spawns *one thread per client*
 - a thread is a part of the program (i.e., a sequence of instructions) that the programmer flags as **runnable in parallel** to other threads

Design 3: multiple threads

- We can re-factor our program so that the OS runs a *single process* which spawns *one thread per client*
 - a thread is a part of the program (i.e., a sequence of instructions) that the programmer flags as **runnable in parallel** to other threads
- Behind the scenes, the OS reserves a few resources to each thread
 - threads can be thought of as lightweight processes, that **share most resources**
 - threads are also **minimal scheduling units**: they run simultaneously, independently, possibly on different cores

Design 3: multiple threads

- We can re-factor our program so that the OS runs a *single process* which spawns *one thread per client*
 - a thread is a part of the program (i.e., a sequence of instructions) that the programmer flags as **runnable in parallel** to other threads



Design 3: multiple threads

- When running the code in production, clients complain that the application has bugs. *Why?*

Design 3: multiple threads

- When running the code in production, clients complain that the application has bugs. *Why?*
- Problem: writing multi-threaded software is much harder than single-threaded one
 - multi-threaded software can have **unexpected interactions with the underlying (performance-optimized) hardware**
 - what if two threads running on different cores want to modify the same variable nearly at the same time?

Core 1	
lw	\$t1,0(\$t0)
addi	\$t2,\$t1,1
sw	\$t2,0(\$t0)

Core 2	
lw	\$t1,0(\$t0)
addi	\$t2,\$t1,1
sw	\$t2,0(\$t0)

Design 3: multiple threads

- When running the code in production, clients complain that the application has bugs. *Why?*
- Problem: writing multi-threaded software is much harder than single-threaded one
 - multi-threaded software can have **unexpected interactions with the underlying (performance-optimized) hardware**
 - in general, **sharing resources** (e.g., variables) creates risks of incorrectness (e.g., think about data hazards in ILP)
 - for programmers, it is much **more challenging to reason about** code executed in parallel

Design 3: multiple threads

- When running the code in production, clients complain that the application has bugs. *Why?*
- Problem: writing multi-threaded software is much harder than single-threaded one
 - multi-threaded software can have **unexpected interactions with the underlying (performance-optimized) hardware**
 - in general, **sharing resources** (e.g., variables) creates risks of incorrectness (e.g., think about data hazards in ILP)
 - for programmers, it is much **more challenging to reason about** code executed in parallel
- Solution: **synchronization and design patterns**
 - ... which is the topic of the coming weeks

Note: design 3 is not always the best one

- The previous slides do ***not*** imply that all server applications ***must*** be multi-threaded
 - design 1 and 2 may be better for applications that are not CPU intensive or not time-sensitive, for example
- In fact, there exists a fundamental **tradeoff between simplicity, correctness and performance**
 - rule of thumb: you often want to target the easiest design that enables to accommodate your requirements
- There are additional designs too!
 - e.g., multi-process with multiple threads per process

Note 2: multithreading $\neq$ true parallelism

- Support to multithreading depends on languages
 - e.g., Python does not support true parallelism by default!

threading — Thread-based parallelism

CPython implementation detail: In CPython, due to the [Global Interpreter Lock](#), only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multi-core machines, you are advised to use [multiprocessing](#) or [concurrent.futures.ProcessPoolExecutor](#). However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

GIL and performance considerations

Unlike the [multiprocessing](#) module, which uses separate processes to bypass the [global interpreter lock](#) (GIL), the threading module operates within a single process, meaning that all threads share the same memory space. However, the GIL limits the performance gains of threading when it comes to CPU-bound tasks, as only one thread can execute Python bytecode at a time. Despite this, threads remain a useful tool for achieving concurrency in many scenarios.

As of Python 3.13, [free-threaded](#) builds can disable the GIL, enabling true parallel execution of threads, but this feature is not available by default (see [PEP 703](#)).

Let's assume we do opt for multithreading

- When running the code in production, clients complain that the application has bugs. *Why?*
- Problem: writing multi-threaded software is much harder than single-threaded one
 - multi-threaded software can have **unexpected interactions with the underlying (performance-optimized) hardware**
 - in general, **sharing resources** (e.g., variables) creates risks of incorrectness (e.g., think about data hazards in ILP)
 - for programmers, it is much **more challenging to reason about** code executed in parallel
- Solution: **synchronization and design patterns**
 - ... which is the topic of the coming weeks

Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**

Stepping stone: the concurrency abstraction

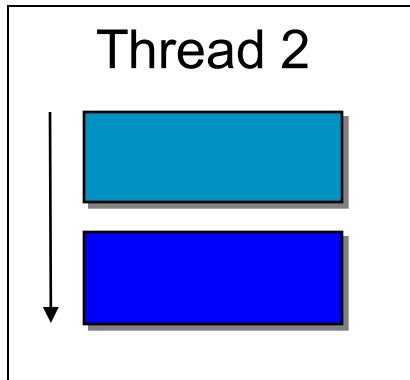
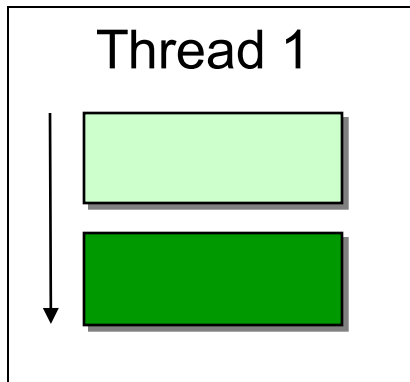
- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**
- Each **thread** corresponds to a totally ordered sequence of atomic actions
 - atomic actions are indivisible in terms of processing

Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**
- Each **thread** corresponds to a totally ordered sequence of atomic actions
 - atomic actions are indivisible in terms of processing
- We thus define **interleaving** as a **totally ordered sequence of executed actions**, across all threads:
 - *only one action at the time* is executed
 - actions from the *same thread* executed in order
 - the overall sequence of executed actions can arbitrarily mix actions from different threads

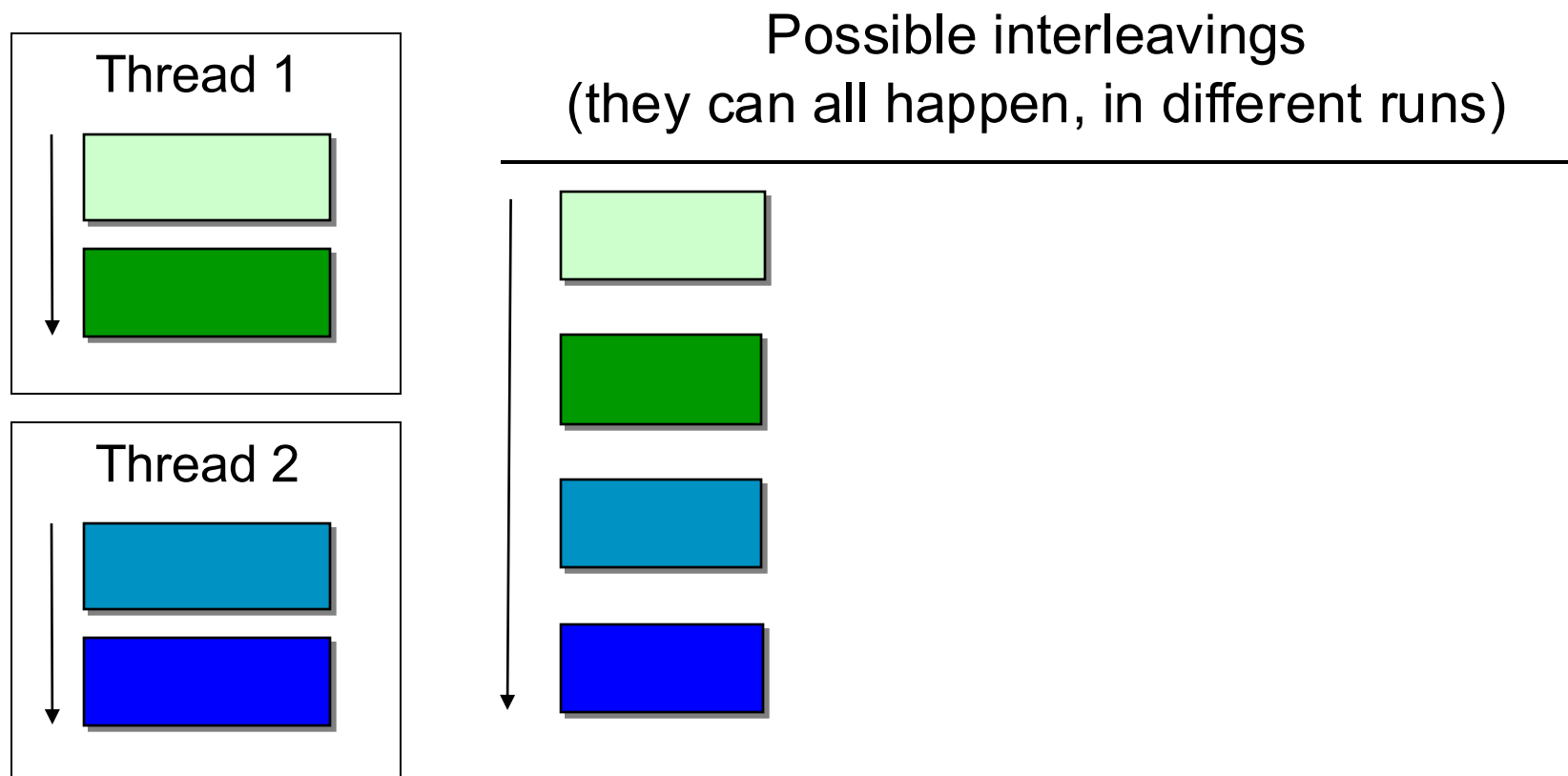
Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**



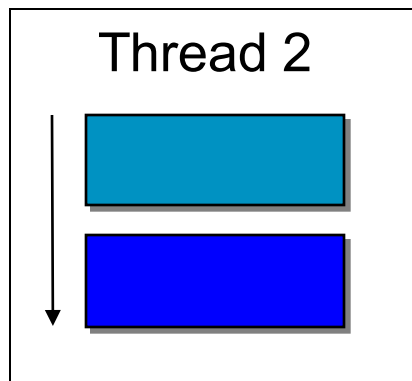
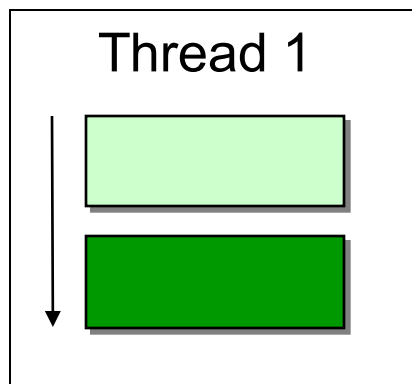
Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**

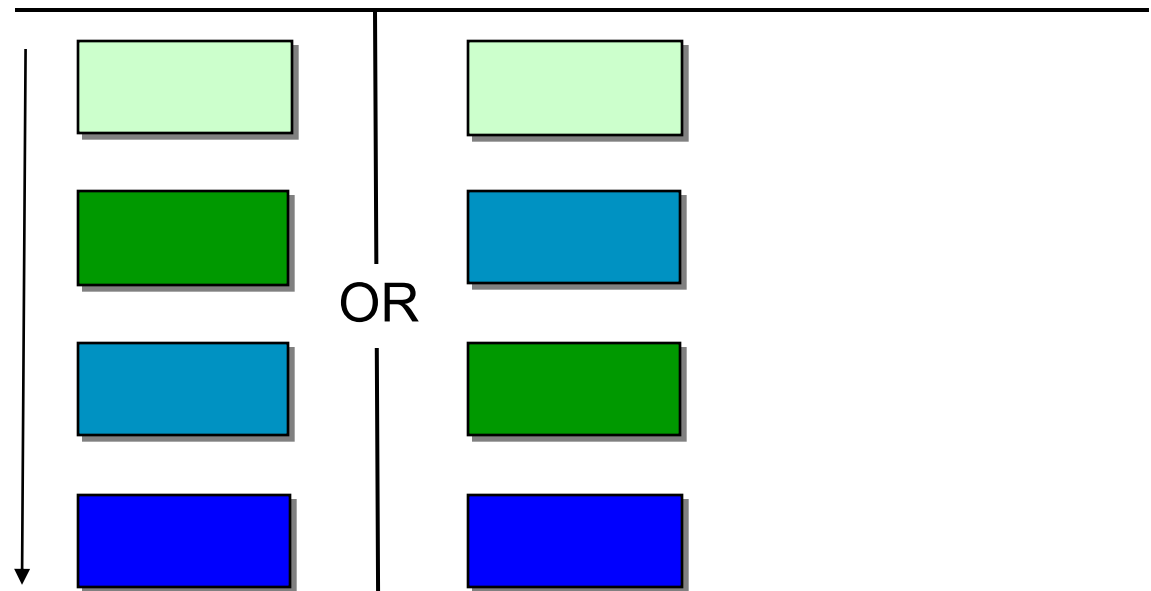


Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**

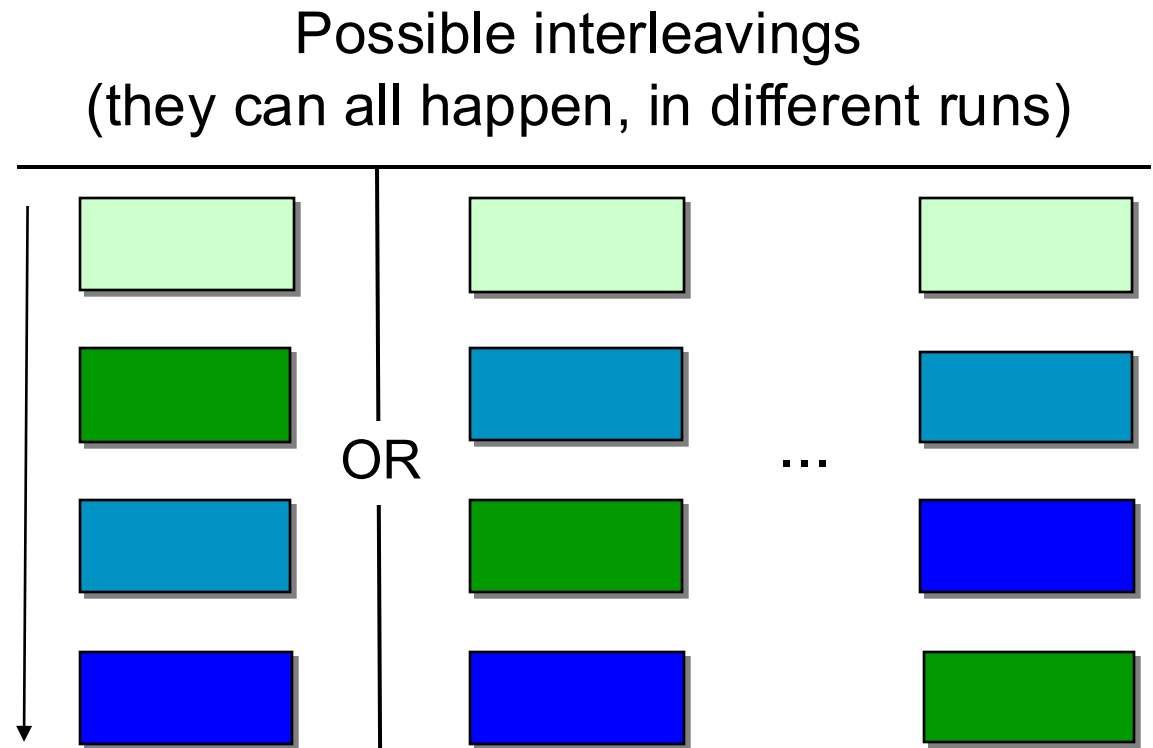
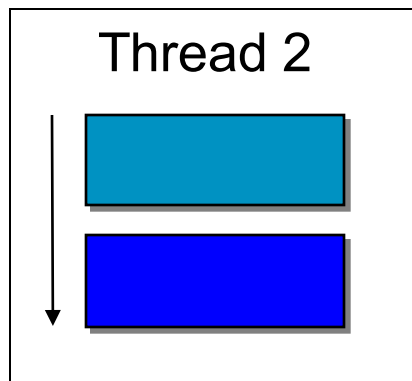
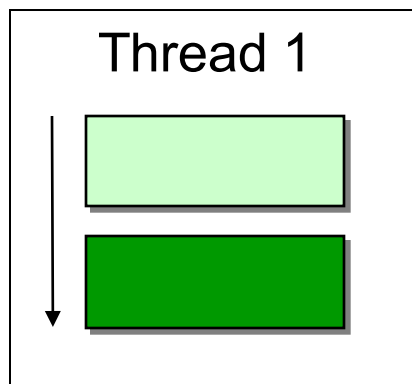


Possible interleavings
(they can all happen, in different runs)



Stepping stone: the concurrency abstraction

- Programmer's viewpoint: the hardware can **unpredictably interleave actions from all threads**



Stepping stone: the concurrency abstraction

- Is this abstraction **correct**?
 - scope: we want to **model problems arising from actions from different threads** (e.g., executed on different cores)
- Concerns
 - we assume a **total order across actions**
 - we **don't model time**
 - we **don't model hardware details**
 - e.g., instruction reordering, caching, thread-to-core allocation, ...

Stepping stone: the concurrency abstraction

- Is this abstraction **correct**?
 - scope: we want to **model problems arising from actions from different threads** (e.g., executed on different cores)
- Reassuring **observations**
 - only one processor at the time can access shared resources, so **hardware effectively imposes a total order**
 - especially about when actions finish
 - hardware also guarantees **correctness per thread**
 - no bad interleaving == no misbehaviour **for any duration of individual actions**
 - NB: we cannot make assumptions on duration of any specific action
 - NB2: **sleep** instructions do NOT solve problematic interleavings!

Building upon the concurrency abstraction

- The concurrency **abstraction is useful** to understand and solve concurrency problems/bugs
 - solutions rely on **primitives of programming languages**
- Programming languages have **constructs** to:
 - define code blocks executed atomically (critical sections)
 - restrict the set of possible interleavings
- Programming languages precisely define the **guarantees** those constructs provide
 - irrespectively of the underlying hardware
- We will study these aspects in Java

Training our intuition: CW4

- Please, connect to www.menti.com
- There, enter code 5207 6485



Please enter the code

Submit

The code is found on the screen in front of you

Training our intuition: two threads

- How many interleavings if we have a thread $A = \{A1, A2\}$ and a thread $B = \{B1\}$?

Training our intuition: two threads

- How many interleavings if we have a thread $A = \{A1, A2\}$ and a thread $B = \{B1\}$?
- *Reminder:* in the concurrency abstraction, we don't reorder actions within threads
 - so $A2$ always follows $A1$
 - other than that, we mix atomic actions in all possible ways
- *Answer:* 3
 - the interleavings are $[A1, A2, B1]$, $[A1, B1, A2]$, $[B1, A1, A2]$

Training our intuition: two threads (variant)

- How many interleavings if we have a thread $A = \{A1, A2\}$ and a thread $B = \{B1, B2\}$?

Training our intuition: two threads (variant)

- How many interleavings if we have a thread $A = \{A1, A2\}$ and a thread $B = \{B1, B2\}$?
- Let's start enumerating
 - if A1 is first: $[A1, A2, B1, B2]$, $[A1, B1, A2, B2]$, $[A1, B1, B2, A2]$
 - If B1 is first: $[B1, A1, A2, B2]$, $[B1, A1, B2, A2]$, $[B1, B2, A1, A2]$
 - no interleaving starts with A2 or B2
- *Answer: 6*
 - note: one more action $\rightarrow$ double the number of interleavings...

Training our intuition: any two threads

- Can you derive a general formula of the number of interleavings for two threads A and B if A has X atomic actions and B has Y actions?

Training our intuition: any two threads

- Can you derive a general formula of the number of interleavings for two threads A and B if A has X atomic actions and B has Y actions?

- This is basically a *stars-and-bars* calculation

★ ★ ★ ★ || ★ | ★ ★ |

- if we have n stars and m bars, the number of possible sequences (including those with consecutive bars) is the binomial coefficient of $n+m$ over n
- *Answer:* $(X+Y)! / X!Y!$

Training our intuition: three threads

- How many interleavings if thread $A = \{A1\}$, thread $B = \{B1\}$ and thread $C = \{C1\}$?

Training our intuition: three threads

- How many interleavings if thread $A = \{A1\}$, thread $B = \{B1\}$ and thread $C = \{C1\}$?
- Let's enumerate them
 - A1 first: $[A1, B1, C1]$, $[A1, C1, B1]$
 - B1 first: $[B1, A1, C1]$, $[B1, C1, A1]$
 - C1 first: $[C1, B1, A1]$, $[C1, A1, B1]$
- *Solution: 6*

Training our intuition: three threads (variant)

- How many interleavings if thread $A = \{A1, A2\}$, thread $B = \{B1\}$ and thread $C = \{C1\}$?

Training our intuition: three threads (variant)

- How many interleavings if thread $A = \{A1, A2\}$, thread $B = \{B1\}$ and thread $C = \{C1\}$?
- Let's start enumerating
 - A1 first: $[A1, A2, B1, C1]$, $[A1, A2, C1, B1]$, $[A1, B1, A2, C1]$, $[A1, B1, C1, A2]$, $[A1, C1, A2, B1]$, $[A1, C1, B1, A2]$
 - B1 first: $[B1, A1, A2, C1]$, $[B1, A1, C1, A2]$, $[B1, C1, A1, A2]$
 - C1 first: $[C1, B1, A1, A2]$, $[C1, A1, B1, A2]$, $[C1, A1, A2, B1]$
- *Solution:* 12
 - twice than if we had 2 threads with 2 actions each...

Training our intuition: simple program

- What are the possible values of x after the termination of $T1: \{x=5; x=2*x\}$ and $T2: \{x=x+2\}$?

Training our intuition: simple program

- What are the possible values of x after the termination of $T1: \{x=5; x=2*x\}$ and $T2: \{x=x+2\}$?
- Let's apply the same methodology as before
 - interleaving1: $x=5; x=2*x; x=x+2; \rightarrow x=12$
 - interleaving2: $x=5; x=x+2; x=2*x; \rightarrow x=14$
 - interleaving3: $x=x+2; x=5; x=2*x; \rightarrow x=10$

Training our intuition: simple program

- What are the possible values of x after the termination of $T1: \{x=5; x=2*x\}$ and $T2: \{x=x+2\}$?
- Let's apply the same methodology as before
 - interleaving1: $x=5; x=2*x; x=x+2; \rightarrow x=12$
 - interleaving2: $x=5; x=x+2; x=2*x; \rightarrow x=14$
 - interleaving3: $x=x+2; x=5; x=2*x; \rightarrow x=10$
- But are all those instructions atomic?
 - $x=2*x$ entails read & write: can be written as $t=x; x=2*t;$
 - same for $x=x+2$, which can be written as $s=x; x=s+2;$

Training our intuition: simple program

- What are the possible values of x after the termination of $T1: \{x=5; x=2*x\}$ and $T2: \{x=x+2\}$?
- But are all those instructions atomic?
 - $x=2*x$ entails read & write: can be written as $t=x; x=2*t;$
 - same for $x=x+2$, which can be written as $s=x; x=s+2;$
- So additional values are possible!
 - interleaving4: $s=0; T1; x=s+2; \rightarrow x=2$
 - interleaving5: $s=0; x=5; x=s+2; x=2*x; \rightarrow x=4$
 - interleaving6: $x=5; s=5; ...; x=s+2 \rightarrow x=7$

Lessons (to be) learned

- There is a **combinatorial explosion** of possible interleavings
 - binomial coefficient with just two threads
 - more threads → more degrees of freedom → even more combinations
- Real concurrent programs may have ***many*** threads, each performing ***many*** actions
- Lesson: it's hard to statically analyse real concurrent program; we **need to design them well**
 - manual analyses and formal tools do not scale

Lessons (to be) learned

- Lesson: combinatorial explosion of interleavings leads to **unpredictability** and **non-determinism** *
 - many possible results, that are even hard to enumerate for large programs
 - some interleavings may be (much) more likely than others: concurrency **bugs are hard to uncover** even with testing!
 - other interleavings can happen (very) rarely → concurrency bugs are **latent!**

* for this reason, we will focus on reading and reasoning on concurrent code in this module

Additional content

- Additional info on threads [GPBBHL 1.1-1.2, Java doc]
 - includes very brief history and other use cases
- Support for multithreading [Moodle]
 - at the hardware and OS (e.g., context switch)

Optionally:

- Cache coherence [P&H 5.8]
 - follow-up of last-week observation on cache coherence in multicore processors